

好的，这是一个非常有趣且经典的问题。这种“交换并记忆化”的方法并非“玄学”，而是一种巧妙的**摊还分析 (Amortized Analysis)** 的应用。它本质上是根号分治思想的一种更自适应、更灵活的实现。

我们来严格证明它的时间复杂度。

## 1. 算法描述

首先，我们明确一下算法的流程：

### 1. 预处理:

- 遍历一遍数组  $A$  (长度为  $n$ )。
- 统计每个数值  $v$  的出现次数  $\text{count}(v)$ 。
- 将每个数值  $v$  出现的所有下标存入一个有序列表  $\text{pos}(v)$ 。
- 这一步的时间复杂度是  $O(n)$ 。

### 2. 查询 $(x, y)$ :

- 规范化:** 定义一个规范序对  $(u, v)$ 。如果  $\text{count}(x) \leq \text{count}(y)$ ，则  $u=x, v=y$ 。否则， $u=y, v=x$ 。
- 记忆化查询:** 检查答案  $\text{ans}(u, v)$  是否已经计算并存储在哈希表  $\text{memo}$  中。
- 计算 (如果未命中缓存):**
  - 如果  $\text{ans}(u, v)$  不在  $\text{memo}$  中，则进行计算。
  - 我们遍历  $\text{pos}(u)$  中的每个下标  $i$ 。
  - 对于每个  $i$ ，在  $\text{pos}(v)$  中二分查找大于  $i$  的下标数量。
  - 将这些数量累加得到  $\text{ans}(u, v)$ 。
  - 计算成本:**  $O(\text{count}(u) * \log(\text{count}(v)))$ 。
  - 将结果存入缓存:  $\text{memo}[(u, v)] = \text{ans}(u, v)$ 。
- 返回结果:**
  - 从缓存中获取  $\text{cached\_ans} = \text{memo}[(u, v)]$ 。
  - 如果查询的原始序对  $(x, y)$  就是规范序对  $(u, v)$ ，则返回  $\text{cached\_ans}$ 。
  - 否则 (说明  $x$  和  $y$  被交换过)，返回  $\text{count}(x) * \text{count}(y) - \text{cached\_ans}$ 。

## 2. 时间复杂度证明

总时间复杂度 = 预处理时间 +  $q$  次查询的总时间。

预处理是  $O(n)$ 。关键是分析所有查询的总计算成本。计算只在“缓存未命中”时发生。因此，总计算成本是**所有被计算过的唯一规范序对  $(u, v)$  的计算成本之和**。

为了分析这个总和的上限，我们引入一个阈值  $B$  来进行论证（注意：这个  $B$  仅用于分析，不存在于算法代码中）。我们将所有被计算过的规范序对  $(u, v)$  分为两类：

- 第一类:**  $u$  的出现次数  $\text{count}(u) \leq B$ 。
- 第二类:**  $u$  的出现次数  $\text{count}(u) > B$ 。

## 分析第一类计算

对于一个规范序对  $(u, v)$ ，如果  $\text{count}(u) \leq B$ ，那么计算它的成本是  $O(\text{count}(u) * \log(\text{count}(v)))$ 。  
这个成本的上限是  $O(B * \log n)$ 。

在  $q$  次查询中，我们最多会计算  $q$  个不同的规范序对。因此，所有属于第一类的计算，其总成本不会超过  $q$  次计算的  
成本上限之和。

$$\text{总成本 (第一类)} \leq q * O(B * \log n) = O(q * B * \log n)$$

## 分析第二类计算

对于一个规范序对  $(u, v)$ ，如果  $\text{count}(u) > B$ ，那么  $u$  是一个“重”元素。

因为  $(u, v)$  是规范的，所以我们有  $\text{count}(v) \geq \text{count}(u) > B$ 。这意味着  $v$  也必须是一个“重”元素。

现在，我们从另一个角度来计算总成本：对每个可能的“重”元素  $u$ ，它作为规范序对的第一个元素，其贡献的总计算成本是多少？

### 1. 重元素的数量:

数组中出现次数超过  $B$  的不同数值（重元素）的数量最多是  $n / B$ 。因为如果每个重元素都至少出现  $B+1$  次，它们的总出现次数不能超过  $n$ 。

### 2. 一个重元素 $u$ 的贡献:

假设  $u$  是一个重元素。它可能与哪些  $v$  组成规范序对  $(u, v)$  并被计算？

$v$  也必须是重元素，且  $\text{count}(v) \geq \text{count}(u)$ 。

最多有  $n / B$  个这样的  $v$ 。

当  $u$  与某个  $v$  配对计算时，成本是  $\text{count}(u) * \log(\text{count}(v))$ 。

因此，对于一个固定的重元素  $u$ ，它贡献的总计算成本上限是：

$$\begin{aligned} & \text{sum}_{\{v \text{ 是与 } u \text{ 配对过的重元素}\}} \text{count}(u) * \log(\text{count}(v)) \\ & \leq \text{count}(u) * \text{sum}_{\{v \text{ 是与 } u \text{ 配对过的重元素}\}} \log n \\ & \leq \text{count}(u) * (n / B) * \log n \text{ (因为最多有 } n/B \text{ 个重元素 } v \text{)} \end{aligned}$$

### 3. 所有重元素 $u$ 的总贡献:

$$\begin{aligned} \text{总成本 (第二类)} & \leq \text{sum}_{\{u \text{ 是重元素}\}} (\text{count}(u) * (n / B) * \log n) \\ & = (n / B * \log n) * \text{sum}_{\{u \text{ 是重元素}\}} \text{count}(u) \end{aligned}$$

$\text{sum}_{\{u \text{ 是重元素}\}} \text{count}(u)$  是所有重元素在数组中的总出现次数，这个值显然不会超过数组总长度  $n$ 。

$$\text{所以，总成本 (第二类)} \leq (n / B * \log n) * n = O((n^2 / B) * \log n)$$

## 寻找最优复杂度

现在我们把两部分的总成本加起来：

$$\text{总计算成本 } T(B) = O(q * B * \log n + (n^2 / B) * \log n)$$

为了让这个总成本最小，我们需要平衡  $q * B$  和  $n^2 / B$  这两项。令它们相等：

$$q * B = n^2 / B$$

$$B^2 = n^2 / q$$

$$B = n / \sqrt{q}$$

我们将这个最优的  $B$  值代回总成本表达式：

$$T = O(q * (n / \sqrt{q})) * \log n + (n^2 / (n / \sqrt{q})) * \log n)$$

$$T = O(n * \sqrt{q} * \log n + n * \sqrt{q} * \log n)$$

$$T = O(n * \sqrt{q} * \log n)$$

## 结论

加上预处理的  $O(n)$ ，该“交换并记忆化”算法的总时间复杂度为  $O(n + n * \sqrt{q} * \log n)$ 。

与标准根号分治的对比:

- 标准根号分治 ( $B=\sqrt{n}$ ):  $O((n+q) * \sqrt{n})$ 。
- 本方法 (自适应):  $O(n + n * \sqrt{q} * \log n)$ 。

在  $n$  和  $q$  同阶的情况下 (例如  $n, q \leq 10^5$ )， $\sqrt{q}$  约等于  $\sqrt{n}$ ， $\log n$  是一个较小的常数因子。两种方法的复杂度级别非常接近，都是  $O(N^{1.5})$  级别，因此都能通过题目。

本方法的优势在于它的**自适应性**。当  $q$  远小于  $n$  时 (例如  $q=1$ )，本方法的复杂度趋向于  $O(n \log n)$ ，而标准根号分治仍为  $O(n * \sqrt{n})$ ，此时本方法更优。反之，如果  $q$  远大于  $n$ ，本方法可能会稍慢一些。但在  $n$  和  $q$  同阶的典型竞赛场景下，两者表现相当。

所以，这个方法绝非“玄学”，它是一个有坚实理论基础、性能优异的摊还分析算法。